XÂY DỰNG AGENT CÓ BỘ NHỚ BỀN VỮNG – CHALLENGE A

Họ và tên: Đào Minh Thiện

MSSV: 23521477

Môn học: SE373.R11

1. Mục tiêu và yêu cầu đề bài

LLM không có trạng thái: mỗi lần gọi API chỉ thấy dữ liệu của request hiện tại. Vì vậy việc "nhớ" phải do harness đảm nhiệm: quyết định lưu gì, truy xuất gì và đưa lại gì vào request kế tiếp. Challenge A yêu cầu một agent nhớ quy ước dự án qua nhiều session độc lập và cập nhật khi quy ước thay đổi, với hai điều phải chứng minh:

- **Chứng minh 1**: session mới (không có chat cũ) vẫn dùng đúng quy ước đã thống nhất.
- **Chứng minh 2**: khi quy ước đổi, các session sau dùng quy ước mới thay vì quy ước cũ.

Ứng dụng viết bằng Python, giao diện HTML/CSS/JavaScript, gọi model thật qua Codex CLI. Memory lưu bằng JSON trên ổ đĩa. Giao diện hiển thị phần memory thực sự được đưa vào context để kiểm chứng.

2. Quyết định thiết kế

Câu hỏi của đề	Lựa chọn	Lý do / đánh đổi
Lưu ở đâu?	File JSON trên ổ đĩa (schema_version + danh sách entries)	Dễ làm, dễ kiểm tra bằng mắt. Đổi lại khó tìm kiếm và khó xử lý xung đột khi dữ liệu lớn; phù hợp phạm vi demo.
Ai ghi?	Agent tự gọi tool remember_memory	Model tự quyết định lúc nào ghi; Python thực thi và lưu file, nên có log hành động và kết quả để kiểm chứng.
Khi nào nạp?	Nạp toàn bộ record active vào context ngay khi mở session	Số quy ước ít nên không cần index hay tìm kiếm. Với memory lớn cần chuyển sang index + topic files (nạp chi tiết khi cần) để tránh context bloat.
Khi quy ước đổi?	Cùng key: record cũ chuyển superseded, record mới active	Không chỉ append (tránh hai quy ước mâu thuẫn cùng hiệu lực) và không xóa dấu vết: giữ nguồn và thời điểm thay thế.
Kiểm chứng?	Hiển thị JSON memory được inject, các bước agent và panel Persistent memory	Chứng minh được thông tin đến từ memory chứ không phải từ chat cũ; có đối chứng tắt memory.

3. Cách hoạt động

Công cụ. read_memory đọc các quy ước còn hiệu lực; remember_memory ghi mới hoặc cập nhật quy ước theo key.

Vòng lặp. Model chọn hành động → Python thực thi tool → model nhận kết quả → tiếp tục hoặc trả lời cuối (final). Mỗi phiên tối đa sáu bước model để tránh chạy vô hạn.

Schema một record. id, key, value, source_session, created_at, status (active hoặc superseded) và superseded_by khi bị thay thế.

Mở session mới. Hệ thống chỉ đọc record active và inject vào context; không gửi lịch sử chat của phiên trước. Memory nằm trên ổ đĩa không tự xuất hiện trong context, nên bước retrieval và injection này là bắt buộc.

Đối chiếu với vòng đời memory trong slide (observe → decide → write → store → search/retrieve → use; update, forget, expire):

Giai đoạn	Trong bài	Trạng thái
Observe / Decide	Model đọc yêu cầu của user và chọn gọi remember_memory	Có
Write / Store	Python ghi record vào JSON trên ổ đĩa	Có
Retrieve / Use	Nạp record active khi mở session và đưa vào context	Có
Update	Record cũ → superseded, record mới → active	Có

4. Kịch bản và kết quả kiểm chứng

Năm phiên độc lập; phiên 2, 4, 5 dùng cùng câu hỏi: "Đề xuất cách triển khai đăng nhập cho dự án này trong 3 gạch đầu dòng."

Phiên	Thao tác	Kết quả quan sát được	Hình
1	Yêu cầu nhớ "authentication dùng JWT"	Model gọi remember_memory, tool trả ok: true, record auth được lưu ở trạng thái active; sau đó model xác nhận đã lưu	1, 2
2	Mở phiên mới, hỏi cách triển khai đăng nhập	Context inject đúng record JWT, chat cũ 0 tin nhắn; câu trả lời dùng JWT (access token ngắn hạn, middleware xác minh chữ ký và thời hạn)	3, 4
3	Đổi quy ước sang session cookie	Record mới (cookie) active; record JWT chuyển sang superseded, giữ nguồn và thời điểm thay thế	5
4	Mở phiên mới, hỏi lại cùng câu hỏi	Câu trả lời dùng session cookie (HttpOnly, Secure, SameSite, hủy session khi đăng xuất), không còn JWT	6
5	Hỏi cùng câu hỏi nhưng tắt memory	Context "Không có memory được inject", hai tool memory bị chặn; agent không biết quy ước xác thực của dự án	7, 8

Thống kê lần chạy: 7 lượt model (phiên 1: 2, phiên 2: 1, phiên 3: 2, phiên 4: 1, phiên 5: 1) và 2 lượt tool ghi memory. Hai phiên ghi/cập nhật đều gọi tool và nhận kết quả thành công trước khi xác nhận. Các phiên 2, 4, 5 chỉ cần một bước final vì memory đã được inject sẵn, không cần gọi tool đọc.

Kiểm thử tự động: 31/31 test đạt (Hình 9), gồm lưu dữ liệu xuyên tiến trình, thay thế quy ước, tạo context mới, vòng lặp tool và xử lý lỗi. Log request, hành động, kết quả tool và câu trả lời lưu trong thư mục evidence/.

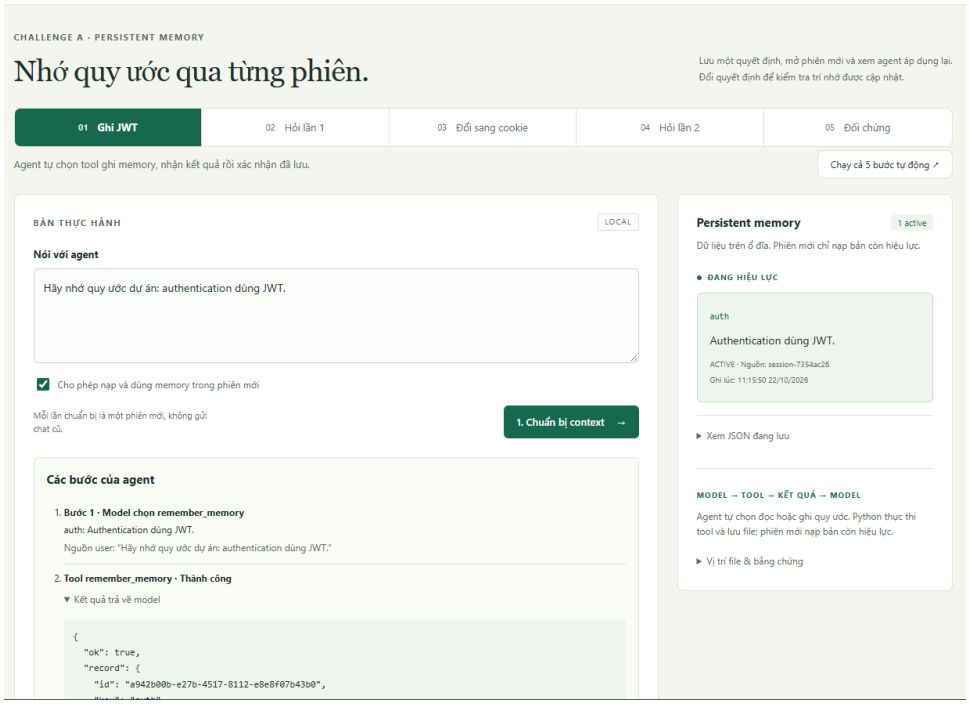
5. Đối chiếu yêu cầu Challenge A

Yêu cầu	Đáp ứng bằng	Bằng chứng
Nhớ quy ước qua nhiều session	Memory JSON trên ổ đĩa, inject khi mở phiên mới	Hình 1–4
Chứng minh 1: session mới dùng đúng quy ước	Phiên 2 trả lời theo JWT với chat cũ = 0 tin nhắn	Hình 3, 4
Chứng minh 2: quy ước đổi thì session sau dùng quy ước mới	Phiên 3 thay thế; phiên 4 trả lời theo session cookie	Hình 5, 6
Xử lý memory lỗi thời	JWT chuyển superseded, giữ nguồn và thời điểm	Hình 5, 6
In ra memory đã inject vào context	Khối "Context của phiên mới" và "Xem toàn bộ prompt"	Hình 3, 7
Đối chứng	Tắt memory: không inject, chặn hai tool	Hình 7, 8

6. Minh chứng chạy chương trình

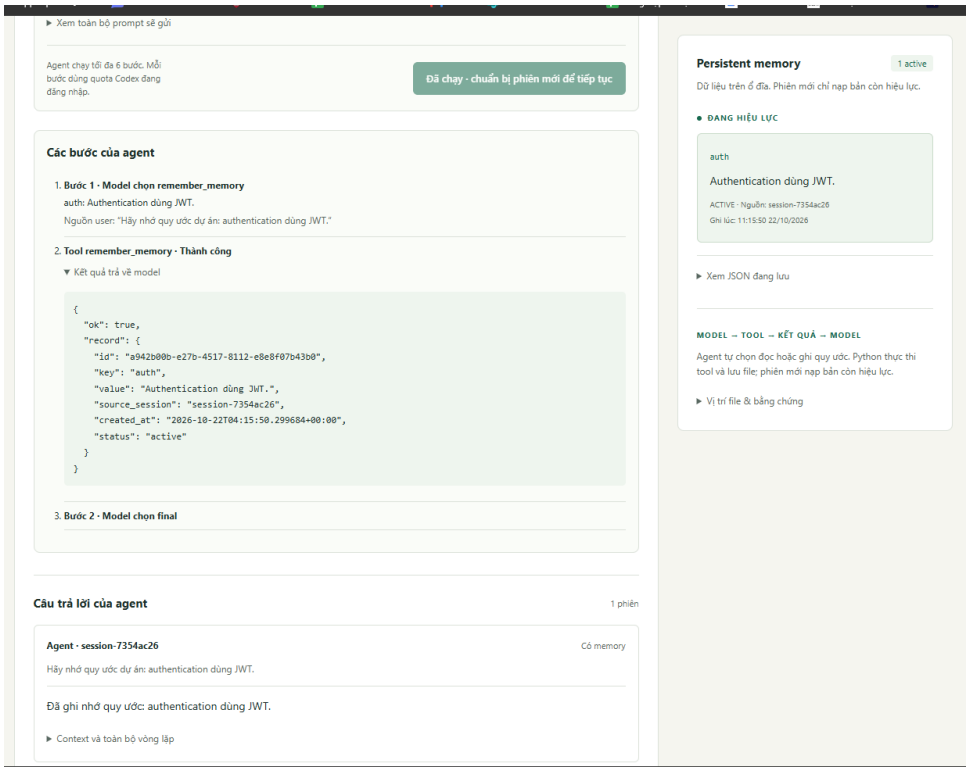
Hình 1 – Phiên 1 – nhập yêu cầu ghi quy ước JWT

- Bàn thực hành, panel Persistent memory (1 active) và các bước agent của phiên 1.



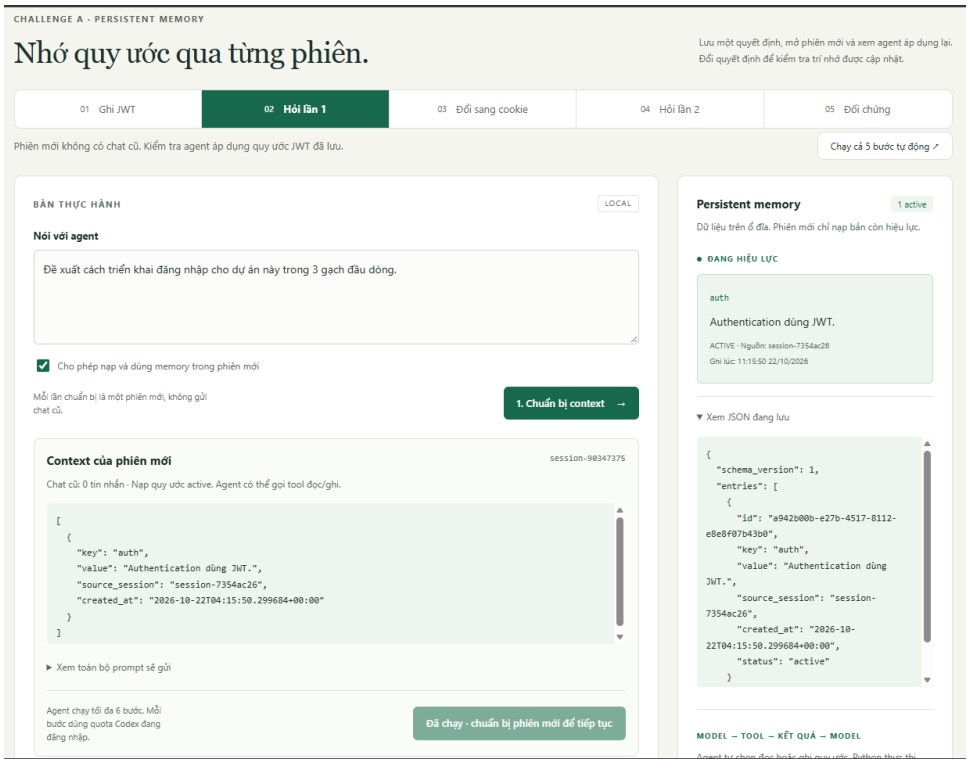
Hình 2 – Agent gọi công cụ lưu quy ước JWT

- Model chọn remember_memory, tool trả ok: true kèm record (id, key, value, source_session, created_at, status), sau đó model chọn final và xác nhận "Đã ghi nhớ quy ước: authentication dùng JWT".



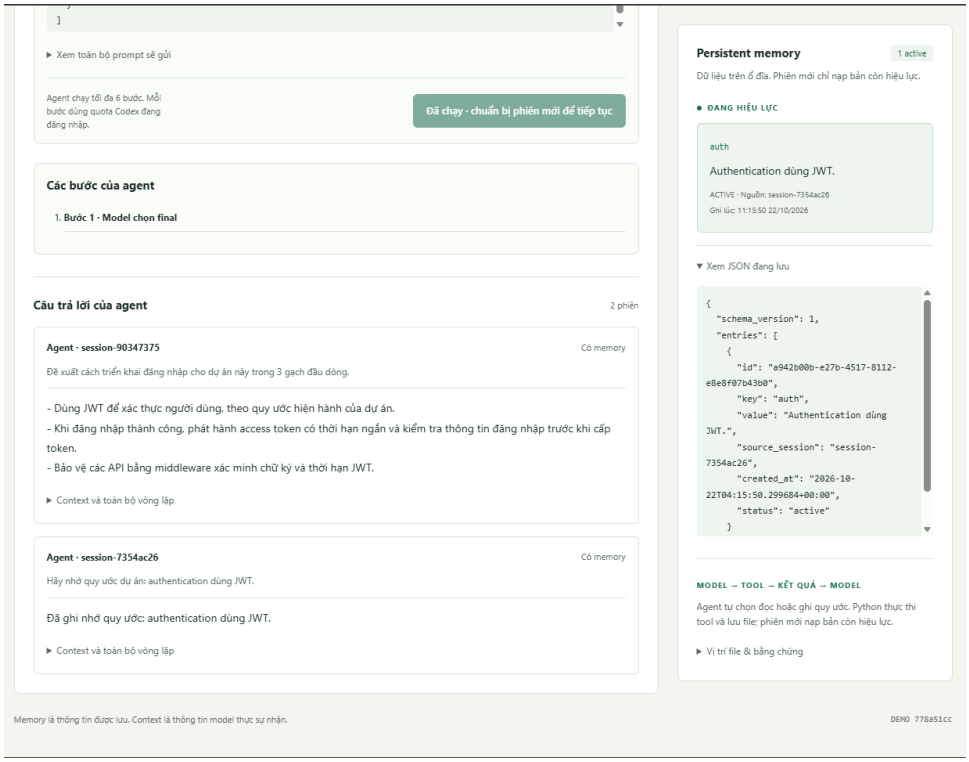
Hình 3 – Phiên 2 – context mới chứa quy ước đã lưu

- Chat cũ: 0 tin nhắn; JSON memory được inject gồm record auth = JWT, nguồn session-7354ac26.



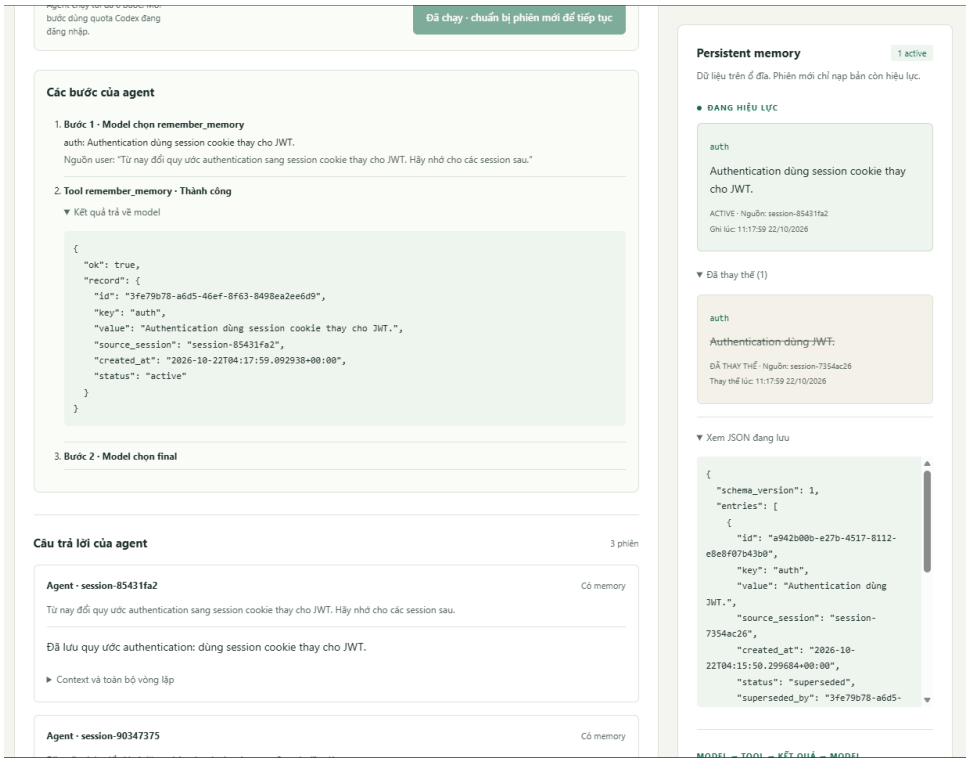
Hình 4 – Phiên 2 – câu trả lời áp dụng JWT

- Hai phiên được liệt kê: câu trả lời của session-90347375 theo JWT (access token ngắn hạn, middleware xác minh chữ ký và thời hạn).



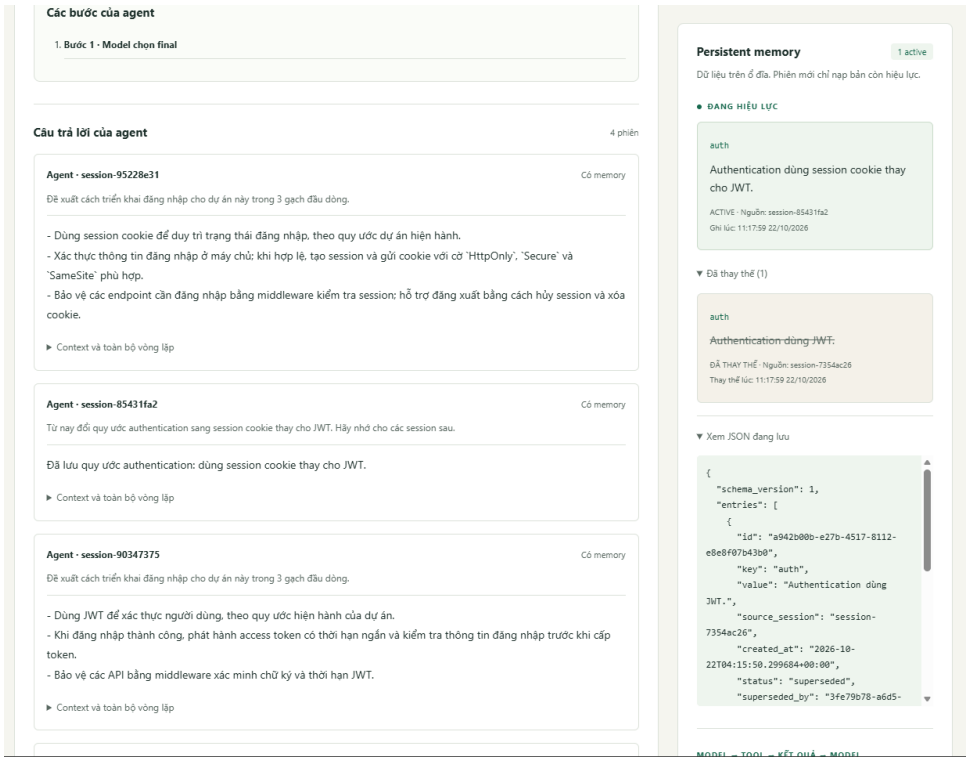
Hình 5 – Phiên 3 – đổi quy ước sang session cookie

- Tool ghi record mới; panel bên phải: cookie là ACTIVE, JWT nằm ở mục “Đã thay thế” (gạch ngang) và JSON có status: superseded, superseded_by.



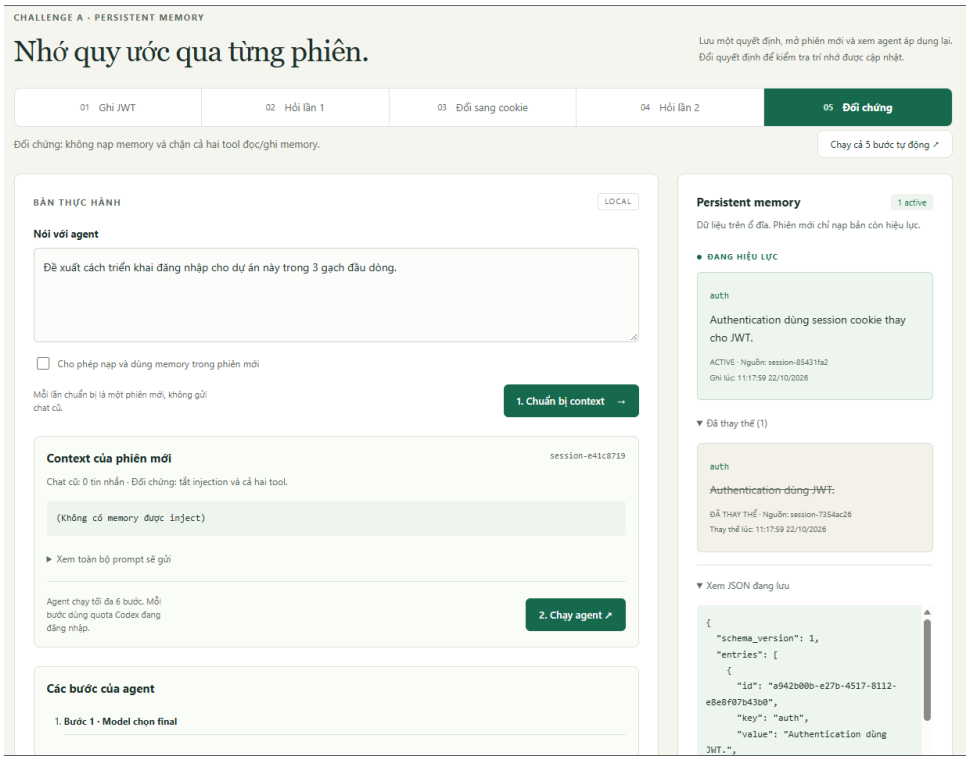
Hình 6 – Phiên 4 – câu trả lời áp dụng session cookie

- Cùng câu hỏi như phiên 2 nhưng câu trả lời mới dùng session cookie (HttpOnly, Secure, SameSite; đăng xuất hủy session và xóa cookie); phiên 2 vẫn giữ nguyên câu trả lời JWT để so sánh.



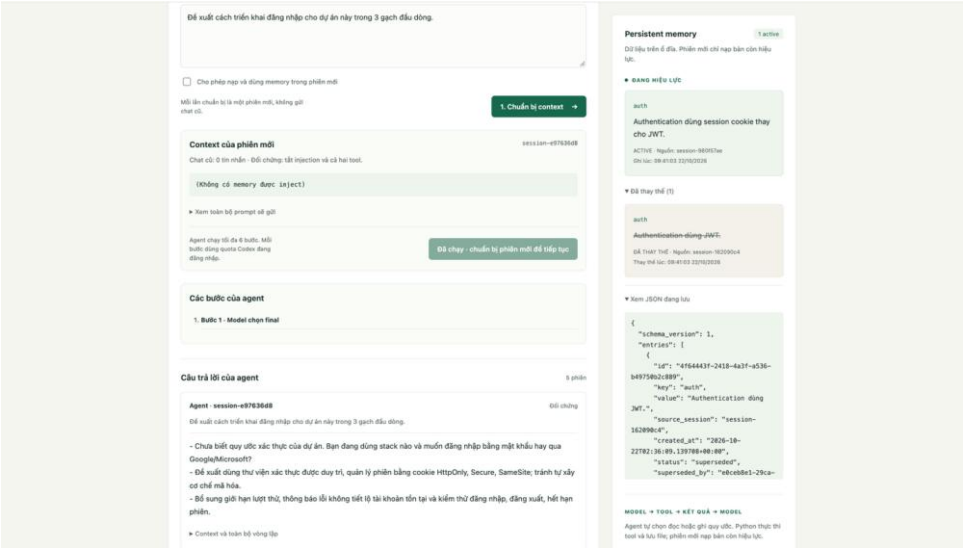
Hình 7 – Phiên 5 – đối chứng: tắt memory

- Ô "Cho phép nạp và dùng memory" bỏ chọn; context ghi "Đối chứng: tắt injection và cả hai tool", nội dung "(Không có memory được inject)", trong khi store trên ổ đĩa vẫn còn quy ước cookie.



Hình 8 – Câu trả lời của phiên đối chứng (lần chạy trước)

- Khi không có memory và không có tool, agent nói chưa biết dự án dùng stack xác thực nào và hỏi lại. Ảnh này lấy từ lần chạy trước (session-e9763648) do ảnh kết quả phiên 5 của lần chạy mới chưa được chụp.



Hình 9 – Kết quả chạy end-to-end bằng demo.py --codex (5 phiên, gồm đối chứng)

- Chạy model thật qua Codex CLI, bằng chứng lưu tại evidence\20261022T042457756840Z.
- Phiên 1 và 3 gọi 1 tool ghi (2 bước model). Phiên 2, 4 và đối chứng chỉ cần 1 bước, 0 tool.
- Phiên 2 trả lời theo JWT, phiên 4 theo session cookie. Phiên đối chứng nói chưa biết stack của dự án nên không đề xuất cụ thể.
- Cuối log có memory_checks = PASS, llm_executed = true và agent_loop_executed = true.

```
PS C:\Users\23521\Downloads\SE373.R11\challenge-a> py -B demo.py --codex
Bằng chứng: C:\Users\23521\Downloads\SE373.R11\challenge-a\evidence\20261022T042457756840Z

session-1-write: 2 bước model, 1 tool
Đã lưu quy ước dự án: Authentication dùng JWT.

session-2-request: 1 bước model, 0 tool
- Dùng JWT để xác thực người dùng, theo quy ước hiện hành của dự án.
- Khi đăng nhập thành công, phát hành access token có thời hạn ngắn và gửi về client.
- Bảo vệ các API bằng middleware kiểm tra chữ ký, hạn sử dụng và quyền trong token.

session-3-update: 2 bước model, 1 tool
Đã cập nhật quy ước authentication: dùng session cookie thay cho JWT.

session-4-request: 1 bước model, 0 tool
- Dùng session cookie để duy trì trạng thái đăng nhập, phù hợp với quy ước hiện hành của dự án.
- Khi đăng nhập thành công, tạo session phía máy chủ và gửi cookie có cờ `HttpOnly`, `Secure` và `SameSite` phù hợp.
- Bảo vệ các endpoint cần xác thực bằng cách kiểm tra session; hỗ trợ đăng xuất bằng cách hủy session và xóa cookie.

control-request: 1 bước model, 0 tool
- Mình chưa biết dự án dùng framework, kiểu ứng dụng hay nhà cung cấp danh tính nào, nên chưa thể đề xuất luồng đăng nhập phù hợp cụ thể.
- Trước tiên, xác định phương thức xác thực cần hỗ trợ (mật khẩu, OAuth/SSO hoặc cả hai) và nơi lưu phiên (cookie bảo mật hoặc token).
- Bạn cho biết stack và yêu cầu đăng nhập của dự án để mình đề xuất cách triển khai sát hơn.
{
  "memory_checks": "PASS",
  "llm_executed": true,
  "agent_loop_executed": true,
  "note": "Đã kiểm tra store, injection và tool trace. Đọc câu trả lời để đánh giá ngữ nghĩa; không chấm việc tuân thủ chỉ bằng từ khóa.",
  "question": "Đề xuất cách triển khai đăng nhập cho dự án này trong 3 gạch đầu dòng.",
  "evidence_dir": "C:\\Users\\23521\\Downloads\\SE373.R11\\challenge-a\\evidence\\20261022T042457756840Z"
}
```

Hình 10 – Kết quả kiểm thử tự động

- python3 -B -m unittest discover -s tests -v: chạy 31 test, trạng thái OK.

```
PS C:\Users\23521\Downloads\SE373.R11\challenge-a> py -3 -B -m unittest discover -s tests -v
>> node --check web/app.js
>> py -3 -B demo.py
test_invalid_tool_and_arguments_can_be_corrected_without_writing (test_agent_loop.AgentLoopTests.test_invalid_tool_and_arguments_can_be_corrected_without_writing) ... ok
test_malformed_model_response_is_rejected_before_writing (test_agent_loop.AgentLoopTests.test_malformed_model_response_is_rejected_before_writing) ... ok
test_model_writes_then_observes_tool_result_before_answering (test_agent_loop.AgentLoopTests.test_model_writes_then_observes_tool_result_before_answering) ... ok
test_new_session_uses_updated_rule_without_previous_transcript (test_agent_loop.AgentLoopTests.test_new_session_uses_updated_rule_without_previous_transcript) ... ok
test_provider_failure_after_write_keeps_actual_record_and_failed_trace (test_agent_loop.AgentLoopTests.test_provider_failure_after_write_keeps_actual_record_and_failed_trace) ... ok
test_read_returns_active_memory_and_final_does_not_write (test_agent_loop.AgentLoopTests.test_read_returns_active_memory_and_final_does_not_write) ... ok
test_step_limit_saves_failed_trace_and_never_reports_success (test_agent_loop.AgentLoopTests.test_step_limit_saves_failed_trace_and_never_reports_success) ... ok
test_timeout_after_model_response_does_not_execute_write (test_agent_loop.AgentLoopTests.test_timeout_after_model_response_does_not_execute_write) ... ok
test_write_requires_quote_from_current_user_not_saved_memory (test_agent_loop.AgentLoopTests.test_write_requires_quote_from_current_user_not_saved_memory) ... ok
test_completed_shell_command_is_rejected (test_codex_events.CodexEventTests.test_completed_shell_command_is_rejected) ... ok
test_startup_warning_is_diagnostic_not_tool_execution (test_codex_events.CodexEventTests.test_startup_warning_is_diagnostic_not_tool_execution) ... ok
test_later_request_loading_the_old_record_is_rejected (test_demo.DemoEvidenceTests.test_later_request_loading_the_old_record_is_rejected) ... ok
test_live_evidence_can_be_verified_after_moving_to_another_machine (test_demo.DemoEvidenceTests.test_live_evidence_can_be_verified_after_moving_to_another_machine) ... ok
test_missing_local_answer_is_not_replaced_by_an_external_original (test_demo.DemoEvidenceTests.test_missing_local_answer_is_not_replaced_by_an_external_original) ... ok
test_new_rule_can_mention_what_it_replaces (test_demo.DemoEvidenceTests.test_new_rule_can_mention_what_it_replaces) ... ok
test_partial_live_run_never_counts_as_completed_demo (test_demo.DemoEvidenceTests.test_partial_live_run_never_counts_as_completed_demo) ... ok
test_changed_rule_supersedes_old_rule_in_later_process (test_memory.MemoryChallengeTests.test_changed_rule_supersedes_old_rule_in_later_process) ... ok
test_control_without_memory_has_same_question_and_no_injection (test_memory.MemoryChallengeTests.test_control_without_memory_has_same_question_and_no_injection) ... ok
test_corrupt_store_is_reported_and_preserved (test_memory.MemoryChallengeTests.test_corrupt_store_is_reported_and_preserved) ... ok
test_empty_value_is_rejected_without_writing (test_memory.MemoryChallengeTests.test_empty_value_is_rejected_without_writing) ... ok
test_new_process_reads_memory_without_old_chat (test_memory.MemoryChallengeTests.test_new_process_reads_memory_without_old_chat) ... ok
test_unchanged_rule_does_not_duplicate_and_other_keys_survive (test_memory.MemoryChallengeTests.test_unchanged_rule_does_not_duplicate_and_other_keys_survive) ... ok
test_background_run_prevents_duplicate_and_exposes_progress (test_ui_server.LabTests.test_background_run_prevents_duplicate_and_exposes_progress) ... ok
test_context_is_frozen_before_model_call_and_cannot_be_sent_twice (test_ui_server.LabTests.test_context_is_frozen_before_model_call_and_cannot_be_sent_twice) ... ok
test_control_omits_memory_and_new_demo_preserves_previous_store (test_ui_server.LabTests.test_control_omits_memory_and_new_demo_preserves_previous_store) ... ok
test_default_runner_uses_agent_loop_and_returns_updated_state (test_ui_server.LabTests.test_default_runner_uses_agent_loop_and_returns_updated_state) ... ok
test_failed_background_run_is_not_retried_and_keeps_store (test_ui_server.LabTests.test_failed_background_run_is_not_retried_and_keeps_store) ... ok
test_http_rejects_foreign_origin_and_handles_local_memory_request (test_ui_server.LabTests.test_http_rejects_foreign_origin_and_handles_local_memory_request) ... Memory Lab: "POST /api/memory HTTP/1.1" 403 -
Memory Lab: "POST /api/memory HTTP/1.1" 200 -
ok
test_invalid_input_never_reaches_model (test_ui_server.LabTests.test_invalid_input_never_reaches_model) ... ok
test_new_session_loads_only_current_rule_and_keeps_same_question (test_ui_server.LabTests.test_new_session_loads_only_current_rule_and_keeps_same_question) ... ok

----------------------------------------------------------------------
Ran 31 tests in 6.128s

OK
Bằng chứng: C:\Users\23521\Downloads\SE373.R11\challenge-a\evidence\20261022T042203957997
{
  "memory_checks": "PASS",
  "llm_executed": false,
  "agent_loop_executed": false,
  "note": "Chỉ kiểm chứng harness offline, không chạy agent/model.",
  "question": "Đề xuất cách triển khai đăng nhập cho dự án này trong 3 gạch đầu dòng.",
  "evidence_dir": "C:\\Users\\23521\\Downloads\\SE373.R11\\challenge-a\\evidence\\20261022T042203957997"
}
```

7. Kết luận

Bài triển khai được việc lưu và cập nhật quy ước qua các phiên độc lập, đồng thời hiển thị memory được nạp và quá trình gọi tool để kiểm chứng. Kết quả đáp ứng hai yêu cầu của Challenge A: phiên mới dùng đúng quy ước đã lưu (JWT), và sau khi quy ước đổi, các phiên sau áp dụng quy ước mới (session cookie) trong khi lịch sử thay thế vẫn được giữ lại. Phiên đối chứng cho thấy hành vi đúng đến từ memory chứ không phải từ chat cũ.